

# VLA 모델(smolVLA, $\pi_0$ ) 비동기 추론 방법론 및 성능 평가 기준 조사

202211135 이동규

## 0. 사전 지식

### 동기 추론:

동기 방식은 '직렬 처리'이다. 앞선 작업이 완전히 끝나야만 다음 작업으로 넘어간다.

- **동작 흐름:** 카메라 관측 → 모델 추론 시작 → (로봇 정지 상태 대기) → 추론 완료 및 Action 생성 → 로봇 행동 실행 → 행동 완료 → 다시 카메라 관측...
- **장점:**
  - 구현이 매우 직관적이고 단순하다.
  - 원인과 결과가 1:1로 매칭되어 디버깅(오류 찾기)이 쉽다.
- **단점 (로봇 제어 시 치명적):**
  - 모델의 추론 시간이 길어질수록 로봇이 멈춰있는 시간(Latency)이 길어진다.
  - 로봇의 움직임이 '뚝뚝 끊기는(Stop-and-go)' 현상이 발생하여 부드러운 제어가 불가능하다.

### 비동기 추론

비동기 방식은 '병렬 처리'이다. 로봇의 물리적 제어(움직임)와 모델의 두뇌 역할(추론)이 서로 다른 스레드/프로세스에서 동시에 돌아간다.

- **동작 흐름:**
  - **제어 루프:** 로봇은 이미 생성된 Action Chunk를 큐(Queue)에서 꺼내 실행 없이 움직인다.
  - **추론 루프:** 로봇이 움직이는 그 순간에도, 카메라는 현재 상태를 찍어 서버로 보내고 다음 Action Chunk를 백그라운드에서 열심히 계산한다.
- **장점:**

- 모델이 다음 행동을 생각하는 시간(추론 지연)을 로봇의 움직임 속에 완벽하게 숨길 수 있다 (Hiding latency).
- 로봇이 멈추지 않고 사람처럼 부드럽고 연속적으로 움직인다.

• 단점:

- 시스템 구조가 복잡해진다. (데이터를 주고받는 통신, 큐 관리, 스레드 동기화 문제 등)
- 새로 도착한 행동(최신)과 기존에 하려던 행동(과거)이 충돌할 때, 이를 어떻게 부드럽게 섞을지(가중 평균 등) 고려해야 한다.

| 구분             | 동기 추론(Synchronous) | 비동기 추론(Asynchronous) |
|----------------|--------------------|----------------------|
| 작업 처리          | 순차적                | 병렬적                  |
| 로봇의 움직임        | Stop-and-go        | Continuous           |
| 추론 지연(Latency) | 로봇의 정지 시간과 직결      | 로봇의 움직임 속에 숨겨짐       |
| 구현 난이도         | 쉬움                 | 복잡함                  |

**Action Chunking:**

일반적인 로봇 제어 모델은 현재 시점  $t$ 의 관측값(카메라 이미지, 로봇 관절 상태 등)  $o_t$ 를 받아 바로 다음 시점의 단일 행동  $a_t$  하나만 예측한다.

$$a_t = \pi(o_t)$$

하지만 Action Chunking은 현재 관측값  $o_t$ 를 바탕으로, 미래의  $k$ 개 step에 해당하는 행동의 묶음(chunk)을 한 번에 예측한다.

$$A_t = [a_t^{(t)}, a_{t+1}^{(t)}, a_{t+2}^{(t)}, \dots, a_{t+k-1}^{(t)}] = \pi(o_t)$$

여기서 윗첨자  $(t)$ 는 시점  $t$ 에서 예측되었음을 의미하며,  $k$ 는 Chunk의 크기(Chunk size)이다.

**도입 이유:** 단일 행동만 예측하면 매 스텝마다 발생하는 작은 오차가 누적되어 로봇이 경로를 이탈하는 문제(Compounding errors)가 발생한다. Chunking을 통해 여러 스텝을 한 번에 묶어서 예측하면, 일관성 있는 궤적을 생성할 수 있어 오차 누적을 크게 줄일 수 있다.

**Temporal Ensembling:**

Action Chunking을 적용하여 매 스텝( $t$ )마다 모델이 계속 추론을 한다고 가정하자. 그럼 로봇

이 특정 시점  $t$ 에서 수행해야 할 행동에 대해, 과거 여러 시점에서 예측한 결과들이 중첩되게 된다.

예를 들어  $k = 3$  일 때, 실제 시점  $t = 3$ 에서 로봇이 해야 할 행동  $a_3$ 에 대한 예측값은 다음과 같이 3개가 존재한다.

1.  $t = 1$  시점에 예측한 3번째 행동:  $a_3^{(1)}$
2.  $t = 2$  시점에 예측한 2번째 행동:  $a_3^{(2)}$
3.  $t = 3$  시점에(지금) 예측한 1번째 행동:  $a_3^{(3)}$

Temporal Ensembling은 이처럼 겹치는 여러 예측값들을 버리지 않고, 가중 평균(Weighted Average)하여 최종 행동  $a_t^{final}$ 을 결정하는 기법이다. 수식으로는 다음과 같이 표현할 수 있다.

$$a_t^{final} = \sum_{i=0}^{k-1} w_i \cdot a_t^{(t-i)}$$

이때 가중치  $w_i$ 는 합이 1이 되도록 설정하며, 일반적으로 지수적 가중치(Exponential weighting)를 사용한다. 가장 최근의 관측 값을 바탕으로 예측한 행동이 가장 정확할 확률이 높으므로, 최근 예측 값에 더 높은 가중치를 부여한다.

$$w_i = \frac{\exp(-m \cdot i)}{\sum_{j=0}^{k-1} \exp(-m \cdot j)}$$

(여기서  $m$ 은 하이퍼 파라미터로,  $m$ 이 클수록 가장 최신 예측 값에 가중치를 몰아준다.)

참고 논문: Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware (Zhao, T. Z., et al., 2023)

## 1. smolVLA 정리 및 비동기 추론 방법론

사전 학습된 VLM과 flow matching으로 학습된 action expert로 구성된 경량화된 VLA. 데이터셋을 이용한 imitation learning으로 먼저 사전 학습된다. 여러 장의 이미지와 작업을 설명하는 language instruction이 주어지면, 모델은 chunk of action을 출력한다. 추론 시에는 비동기 실행 스택을 도입하여, 부드러운 제어를 가능하게 한다.

## 비동기 추론 - Algorithm 1(Aggregated Ensembling)

로봇과 추론 서버를 완벽히 분리함으로써 행동 예측( $A$ )을 행동 실행과 분리한다.

**작동원리:** 로봇이 큐(Queue)에 남아있는 행동을 소비하는 도중에, 임계값( $g$ ) 이하로 큐가 줄어들면 새로운 관측치(카메라 이미지 등)  $o_{\{t+1\}}$ 을 캡처하여 원격 정책 서버(PolicyServer)로 보낸다.

**통합 (Aggregation):** 서버에서 새로운 행동 chunk  $\tilde{A}_{t+1}$ 이 도착하면, 이를 겹치는 시간대(overlapping)에 대해 기존의 큐  $A_t$  통합(aggregate)한다. ( $A_{t+1} \leftarrow f(A_t, \tilde{A}_{t+1})$ ) 과거의 chunk를 버리지 않고 현재 시점과 가까운 행동에 가중치를 주는 방식으로 궤적을 섞어준다.

**임계값  $g$ 에 따른 성능 차이:**

1. **순차적 한계 ( $g = 0$ ):** 큐를 100% 다 소모한 뒤 새 관측을 보냅니다. 새 청크가 계산되는 동안 로봇은 멈춰 있어야 한다.
2. **비동기 추론 ( $g = 0.7$ ):** 기존 큐를 약 30% 정도 썼을 때 다음 관측치를 보내 미리 계산을 시작한다. 계산되는 동안에도 로봇은 남은 70%의 큐를 사용하며 움직이므로 빈틈이 생기지 않는다. 연속된 chunk 사이의 겹치는 부분은 버퍼 역할을 하여 로봇을 부드럽게 제어한다.
3. **컴퓨팅 집약적 한계 ( $g = 1$ ):** 매 스텝마다 계속 관측치를 보내어 반응성은 최고지만, 하드웨어에 엄청난 부하를 준다.

참고 논문: SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics

## 2. $\pi_0$ 정리 및 비동기 추론 방법론

다양한 데이터 소스를 통합하기 위해 사전 학습된 VLM(PaliGemma)을 기반으로 하여 인터넷 규모의 경험, 일반 지식, 의미론적 추론 능력을 그대로 가져온다. 완전히 분리된 Action expert 를 도입해서 모델을 VLA 모델로 변환한다. 여러 로봇 유형의 데이터를 하나의 모델에 통합하는 cross-embodiment 학습을 적용한다. 연속적인 행동 분포를 생성하기 위해 플로우 매칭(디퓨전의 변형)과 함께 Action chunking 아키텍처를 사용한다.

## 비동기 추론 – RTC(Real-time chunking) 알고리즘

이전 chunk 를 실행하면서 동시에 다음 청크를 미리 계산하는 환경 제안한다. 기존 smolVLA 와의 차이는 inpainting 기법이다. (다른 문서에 정리)

### 3. 실험환경에 따른 벤치마크 기준 분리

실제 로봇 제어 환경(우리의 환경)에서는 비동기 추론으로 모델들을 비교하는 것이 적합하다. 실제 물리적 환경에서는 로봇의 시간이 멈추지 않으므로, 추론 지연이 발생하면 로봇이 기존에 하던 작업을 완벽하게 수행하지 못하는 문제가 발생하는 점을 문서에서 발견하였다. 또한 모델의 판단 능력만을 평가하는 것이 아니기에 제어 루프가 깨지는 현상도 발생하는 것을 알 수 있다.

ALOHA/ACT,  $\pi_0$ , AsyncVLA 등의 실제 물리 로봇(Real-world) 배포 및 검증 연구 논문들을 읽어보면 모두 백그라운드에서 추론을 돌리고 로봇은 기존 버퍼의 행동을 수행하는 **비동기 추론(Asynchronous Inference) 환경**을 필수적인 벤치마크 기준으로 삼고 있다.

결론적으로 동기와 비동기 모두 고려하는 것이 바람직하지만 우리 프로젝트 환경 특성상 비동기 추론으로 비교를 하여야 적합할 것 같다고 생각된다.

참고문서:

<https://www.alphaxiv.org/abs/2602.18397> (모델 성능을 평가할 때 추론 지연 처리 기준)

<https://arxiv.org/html/2606.15285v1#:~:text=We%20propose%20an%20asynchronous%20semantic%2Daction%20decoupling%20framework,vision%2Dlanguage%20backbone%20or%20introducing%20an%20external%20planner>. (제어루프가 깨지는 현상 발생)

<https://www.mdpi.com/2313-7673/11/5/316#:~:text=Action%20Chunking%3A%20To%20improve%20temporal%20coherence%20and,%3A%20o%20t%20%E2%86%92%20%7B%20a%20t> (ACT 실험 환경 세팅 방법론 추후 보고서로 정리 예정)